jobops

End-to-End User Guide

A self-hosted MCP server for the full job-search loop — portal scanning, JD evaluation, tailored resume + cover PDFs, outreach drafting, story bank, negotiation brief.

The chat reasons. The server executes. Every artifact comes back as a localhost link.

Version `0.12.0`

Repo https://github.com/mohith-das/jobops

License MIT

Contents

1 What this is, and the mental model

`jobops` is a self-hosted **Model Context Protocol** server that exposes the full job-search workflow to your chat client as 51 tools and 6 editable behaviour-shaping resources. The chat client (Claude Desktop, LibreChat, Cursor, any MCP-aware client that speaks streamable-HTTP) does the reasoning; the server executes the mechanical work and returns every generated artifact as an `http://localhost:7891/...` link the chat can paste straight into the conversation.

The split is the whole point.

The chat thinks.
Reads your rubric and your career packet. Scores a JD. Drafts the 6-block evaluation report. Picks bullets per role type. Writes the warm-intro DM. Decides what to ship.

The server executes.
Persists jobs to a single SQLite file with a known schema. Polls Greenhouse / Ashby / Lever / Workday APIs. Renders Chromium HTML→PDF. Validates outreach drafts against safety rails before saving. Serves every artifact at a stable local URL.

You decide and send.
Nothing auto-submits an application or auto-sends a DM. `jobops` is preview-only at every irreversible boundary.

Where it comes from

`jobops` merges two systems into one process. The 6-block evaluation report, the ATS-friendly HTML CV template, the unicode-normalization pipeline, and the negotiation playbook framing are ported from santifer/career-ops (MIT licensed). The three-dimension rubric (resume / taste / visa), the schema shape, the strict-JSON rater contract, the warm-intro and founder DM tone rules, and the H1B signal logic are distilled from a personal Postgres + n8n pipeline ("JSA").

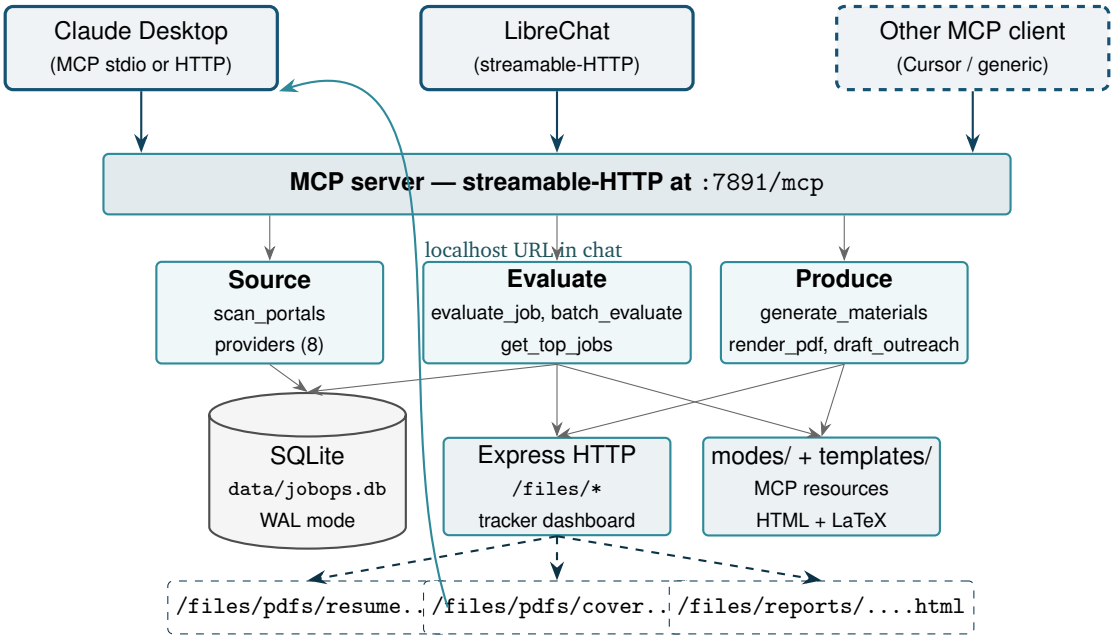
> **Not affiliated with or endorsed by santifer's career-ops.** This is an independent project that adapts the publicly-released MIT-licensed templates and rubric shape into the MCP transport surface. Credit where due, but no endorsement implied.

One sentence

Source jobs cheaply, evaluate them carefully, produce tailored materials, draft (don't send) the outreach, track everything in one local database, do it every day.

2 Architecture

One process. Three planes. Six MCP resources and 51 tools. In HTTP mode that one process serves *many* concurrent MCP clients at once against the one SQLite DB — the shared "one server, every client" topology described in the setup chapter.



Read it like this: the chat client posts a JSON-RPC `tools/call` to `/mcp`. The server routes to one of the three planes — **Source** (pull jobs from ATS APIs), **Evaluate** (score, write the 6-block report, persist to SQLite), or **Produce** (tailor bullets, render PDFs, draft outreach). The Express HTTP server on the same port serves every generated file at `/files/...` and a read-only tracker dashboard at `/`. The chat pastes those links back into the conversation — the human clicks to inspect.

Why one process

No external Postgres. No n8n. No cloud anything. The single-process design means:

- **Boot in 2 seconds.** `npx @mohith_das/jobops start` — no docker compose, no migrations to run by hand, no init container.
- **Local data, local artifacts.** Your jobs, your scores, your LinkedIn export, your H1B import, your tailored PDFs all live next to your `cv.md` in one directory. Back it up by copying that directory.
- **Chat-driven, not autonomous.** The server reacts to chat tool calls. It is not a background daemon scraping the web.
- **Opt-in cron.** If you do want background scans, `scheduler_enable` turns on a per-job interval. Off by default.

3 Installation and setup

Prerequisites

- **Node.js $\geq$ 20.** Check with `node -version`. Install from `https://nodejs.org/` if missing.
- **An MCP-aware chat client.** Claude Desktop, LibreChat, Cursor, or any other client that supports streamable-HTTP MCP.

- **(Optional)** an LLM API key — Gemini (free tier covers ordinary use) or DeepSeek. Required only for the `api/batch` paths; the default `chat` mode of every tool works without one.

Five commands

```
# 1. Scaffold cv.md, config/profile.yml, portals.yml in the current directory
#    and create the SQLite database. Idempotent --- safe to re-run.
npx @mohith_das/jobops init

# 2. Edit the three files. Replace every <TODO> placeholder.

# 3. Rebuild the career_packet from your now-real cv.md. (Or just re-run
#    `init` --- it auto-reseeds when it detects cv.md changed.)
npx @mohith_das/jobops reseed

# 4. Confirm everything is wired.
npx @mohith_das/jobops doctor

# 5. Boot the server. Chromium auto-installs on first run (~150 MB, one-time).
npx @mohith_das/jobops start
```

TIP

The package is scoped (`@mohith_das/jobops`) but the installed command is just `jobops` — once it's on your `PATH` (global install) you can drop the `npx @mohith_das/jobops` prefix and run `jobops start`, `jobops doctor`, etc. directly.

TIP: Editing the packet — two safe directions

The `career_packet` is the source of truth the chat uses to score JDs and draft materials. You can edit it from **either** side, and neither silently destroys the other:

- **Chat-driven** — ask the chat to change a tagline / remove a project / tighten a bullet; it calls `update_career_packet`, which writes a new version and marks the packet *user-edited*. Section edits (`section+section_content`) avoid re-sending the whole packet.
- **File-driven** — edit `cv.md` / `config/profile.yml`, then `reseed`.

`reseed` writes a NEW active version (previous demoted, history kept). To reconcile, `sync_packet_to_cv` writes the packet back into `cv.md` + `config/profile.yml`. Symmetric: `reseed` = cv.md → packet; `sync_packet_to_cv` = packet → cv.md.

SAFETY: Reseed is non-destructive by default

If the active packet has chat edits (`update_career_packet`, `origin=chat_edit`) that are not in `cv.md`, a plain `reseed` **refuses** and warns rather than overwriting them — pass `force` (`reseed --force` / `force:true`) to rebuild from `cv.md` anyway, or run `sync_packet_to_cv` first to write the edits back so a reseed *reproduces* them. `doctor` reports a chat-edited packet as expected ("ahead of cv.md"), not a staleness nag. Standing policy with no CV/profile field (naming conventions, rendering rules, custom guardrails) lives in `modes/career_packet.md` Section 9, which reseed always preserves. A self-contained operator reference ships at `docs/PROJECT_MEMORY.md` — drop it into your MCP client's project memory.

What `init` creates

After `npx @mohith_das/jobops init` from a clean directory you'll have:

```
my-job-search/
|-- cv.md                  <- copy of cv.example.md
|-- config/
|   `-- profile.yml        <- copy of config/profile.example.yml
|-- portals.yml            <- copy of portals.example.yml
`-- data/
    `-- jobops.db         <- SQLite with both migrations applied
```

The three editable files are the "user layer." The server reads them at runtime; you own their content.

`cv.md`

Plain Markdown. Standard headings (`## Work Experience`, `## Projects`, `## Education`, `## Skills`). The renderer parses your bullets into structured experience entries. Keep bullets concrete (action verbs, real metrics).

`config/profile.yml`

Your identity, target roles + archetypes, narrative likes / dislikes, comp range, location, and visa status. The rubric reads this on every score, the materials generator reads it to populate the cover-letter header, and the warm-intro drafter pulls credibility markers from `narrative.superpowers`.

`portals.yml`

The companies whose ATS endpoints `scan_portals` should poll, plus title and location filters. Greenhouse / Ashby / Lever are auto-detected from any `careers_url`; Workday needs an explicit `workday_url`; Amazon / Google / generic Playwright are opt-in via `provider:`.

Environment variables

All optional. Defaults in parens.

Variable	Purpose
JOBOPS_PORT	HTTP port. *(7891)*
JOBOPS_HOST	Bind host. *(127.0.0.1)*
JOBOPS_PROJECT_ROOT	Where cv.md / profile.yml / portals.yml live. *(current directory)*
JOBOPS_DATA_DIR	SQLite location. *(<project_root>/data)*
JOBOPS_OUTPUT_DIR	Where generated PDFs and report HTML land. *(<project_root>/output)*
JOBOPS_VISA_SCORING	Set to false to drop the visa surface entirely (see §8). *(true)*
JOBOPS_PUBLIC_BASE_URL	Public URL emitted in artifact links. Set when running on a remote host (Tailscale / LAN / cloud) so links are reachable from your chat machine. See §3. *(default: http://127.0.0.1:<port>)*
JOBOPS_LLM_PROVIDER	gemini | deepseek | none. *(gemini)*
JOBOPS_LLM_MODEL	Provider-specific model id. *(empty → provider default)*
GEMINI_API_KEY	Gemini key. Needed only for api/batch paths.
DEEPSEEK_API_KEY	DeepSeek key. Same.
JOBOPS_SCHEDULER_ENABLED	Whether opt-in cron runs at all. *(false)*

Connecting your chat client(s): one server, every client

The recommended topology is **ONE long-running HTTP server = ONE source of truth**. Run `npx @mohith_das/jobops start` once (locally, or on an always-on host over Tailscale) and point *every* interface at the same `http://127.0.0.1:7891/mcp` endpoint: Claude Code (`claude mcp add -transport http`), Claude Desktop (via the `mcp-remote` stdio→HTTP bridge, or a paid-plan custom connector), opencode (`opencode.json`, `type: remote`), codex (`~/.codex/config.toml`, `url` + `bearer_token_env_var`), gemini-cli (`settings.json`, `httpUrl`), LibreChat (`streamable-http`), and the tracker web UI. Because there is exactly one process and one SQLite DB behind all of them, materials, tracker moves, contacts, and packet edits made in any client are instantly visible in all the others — when one client is rate-limited, switch to another and continue where you left off.

`npx @mohith_das/jobops connect` prints copy-paste-ready config for each of those clients (flags: `-host`, `-port`, `-token`). Concurrency is safe by design: every HTTP request gets an isolated MCP protocol instance, reads run concurrently (SQLite WAL), and all writes are serialized through one write lock in the single server process — no per-client spawn, no port conflicts, no corruption under multi-client load.

Verify all clients hit the same instance with `npx @mohith_das/jobops status` (flags: `-url`, `-token`): it reports uptime, the source-of-truth DB path + a short fingerprint, and which MCP clients have connected since boot. The `doctor` tool reports the same server-identity block from inside any chat.

Auth: the moment the server is reachable beyond localhost it MUST run with `JOBOPS_AUTH_TOKEN` set (§10) — it serves PII to every connected endpoint and refuses to boot remotely without the token. The token goes into each client's config; `connect` embeds it automatically when the env var is set.

Known issue (mcp-remote × Node ≥ 26): `mcp-remote` (≤ 0.1.38) fails under Node 26+ with `Unexpected content type: null` — its bundled undici `EnvHttpProxyAgent` global dispatcher strips response headers from Node's built-in `fetch`. The server's responses are spec-correct; the bridge mangles them client-side. Until fixed upstream, run the bridge under Node ≤ 24: replace `"command": "npx"` with an absolute path to a Node 24 binary and point `args` at a Node-24-

installed `mcp-remote`. The symptom appears in Claude Desktop's `mcp-server-*.log` right after `Using transport strategy: http-first`.

Below are the two configs you most likely want to paste by hand. The stdio config is the *single-client* alternative: it spawns a private server inside Claude Desktop rather than connecting to the shared one (next to a running shared server it also needs its own `JOBOPS_PORT`, or its file server fails with `EADDRINUSE`).

Claude Desktop (stdio transport — single-client alternative)

Claude Desktop's local MCP only speaks **stdio**, not HTTP — the `-stdio` flag binds MCP to stdin/stdout. The HTTP file server still binds to `JOBOPS_PORT` in the background so the `/files/*` artifact links the chat returns still resolve in your browser.

Add to `~/Library/Application Support/Claude/claude_desktop_config.json` on macOS (or `%APPDATA%/Claude/claude_desktop_config.json` on Windows):

```
{
  "mcpServers": {
    "jobops": {
      "command": "npx",
      "args": ["-y", "jobops", "start", "--stdio"],
      "env": {
        "JOBOPS_PORT": "7891",
        "JOBOPS_PROJECT_ROOT": "/absolute/path/to/your/job-search/dir"
      }
    }
  }
}
```

Restart Claude Desktop. The 48–51 tools should appear in the tool picker.

LibreChat (host process)

In `librechat.yaml`, under top-level `mcpServers:`:

```
mcpServers:
  jobops:
    type: streamable-http
    url: http://127.0.0.1:7891/mcp
    timeout: 60000
```

Boot the server separately (`npx @mohith_das/jobops start`), then restart LibreChat.

LibreChat (Docker)

Inside a container, `127.0.0.1` points at the container itself — not your host. Two changes are required:

```
mcpServers:
```

```
  jobops:
    type: streamable-http
    url: http://host.docker.internal:7891/mcp
    timeout: 60000

mcpSettings:
  allowedAddresses:
    - host.docker.internal:7891
```

SAFETY: Docker SSRF allowlist

LibreChat blocks private / internal addresses by default as SSRF protection. Without the `mcpSettings.allowedAddresses` entry, your MCP server will appear "connected" but every tool call will silently fail.

On Linux, `host.docker.internal` doesn't resolve out of the box. Either add `extra_hosts: ["host.docker.internal:host-gateway"]` to the LibreChat service in your `docker-compose.yml`, or swap the URL for your host's LAN IP and allowlist that IP:port instead.

Running on a remote host / Tailscale

By default every artifact link the server returns starts with `http://127.0.0.1:<port>`. That's fine when you run server + chat on the same machine. If you run the server on a cloud instance, a homelab box, or anything you reach over Tailscale / LAN / a tunnel, `127.0.0.1` on the link resolves to your *chat machine* — not the server — and the links don't work.

Set `JOBOPS_PUBLIC_BASE_URL` to the URL the chat machine actually uses to reach the server:

```
# Tailscale magic DNS
export JOBOPS_PUBLIC_BASE_URL="https://jobs.example.ts.net"

# Tailscale 100.x IP
export JOBOPS_PUBLIC_BASE_URL="http://100.64.0.5:7891"

# LAN IP
export JOBOPS_PUBLIC_BASE_URL="http://192.168.1.20:7891"

# Reverse proxy
export JOBOPS_PUBLIC_BASE_URL="https://jobs.example.com"
```

Every artifact link (resume PDF, .tex, .docx, eval report, apply_prefill screenshot, tracker URL) now uses that base. No other behaviour changes — the server still binds to `JOBOPS_HOST` (default `127.0.0.1`); to accept connections from other devices, also set `JOBOPS_HOST=0.0.0.0`. `npx @mohith_das/jobops doctor` prints the effective public base URL so you can confirm what your links will look like. A malformed value is rejected at boot with a warning on stderr; the server keeps running with the default 127.0.0.1 base. Trailing slashes are stripped.

SAFETY: Exposing beyond localhost

This setting only changes the URLs the server emits. It does NOT add authentication and does NOT change the bind address. If you set `JOBOPS_HOST=0.0.0.0` so the server accepts remote connections, the HTTP file server then serves your resume PDFs, cover letters, eval reports,

LinkedIn-derived data, and H1B-derived data **without authentication** to anyone who can reach the port. Restrict access to a private network (Tailscale, a VPN, a reverse proxy with auth) — never expose this directly on the public internet.

Optional: server-side scoring (BYO key on most clients)

The default behavior of every reasoning tool is `mode: chat` — the server returns the rubric + context, your chat client does the thinking, you call the tool again to persist. No LLM key needed.

For *server-side* scoring (`batch_evaluate`, or `mode: api` on tools that accept it), the server will use **MCP sampling** — asking your connected client's model directly, with **no API key** — *but only if that client advertises the `sampling` capability*. Most clients, **including Claude Desktop as of now, do NOT**, so in practice you set a BYO key for server-side scoring (see §9 and modelcontextprotocol.io/clients). To configure the BYO key:

```
export JOBOPS_LLM_PROVIDER=gemini
export GEMINI_API_KEY=...       # free tier covers ordinary single-user usage

# or
export JOBOPS_LLM_PROVIDER=deepseek
export DEEPSEEK_API_KEY=...

# Force the BYO-key path even when the client could sample:
export JOBOPS_SAMPLING=false
```

`cost_estimate` reports cumulative spend per provider per tool; sampling calls are flagged client-borne ($0 server cost).

Optional: data imports for the outreach + visa features

These are *fully optional*. The system works without them.

- **LinkedIn network** — download your data export at LinkedIn → Settings → Data Privacy → Get a copy of your data → Connections. Call `import_linkedin` with the path to `Connections.csv`. Unlocks `find_warm_intros` and `find_founders`. No CSV? Use `add_contacts` to add people one batch at a time from chat — same store, same discovery.
- **H1B visa signal** — download a DOL OFLC quarterly LCA disclosure CSV from dol.gov. Call `import_h1b`. Unlocks `visa_signal` (returns a friendliness band per company).

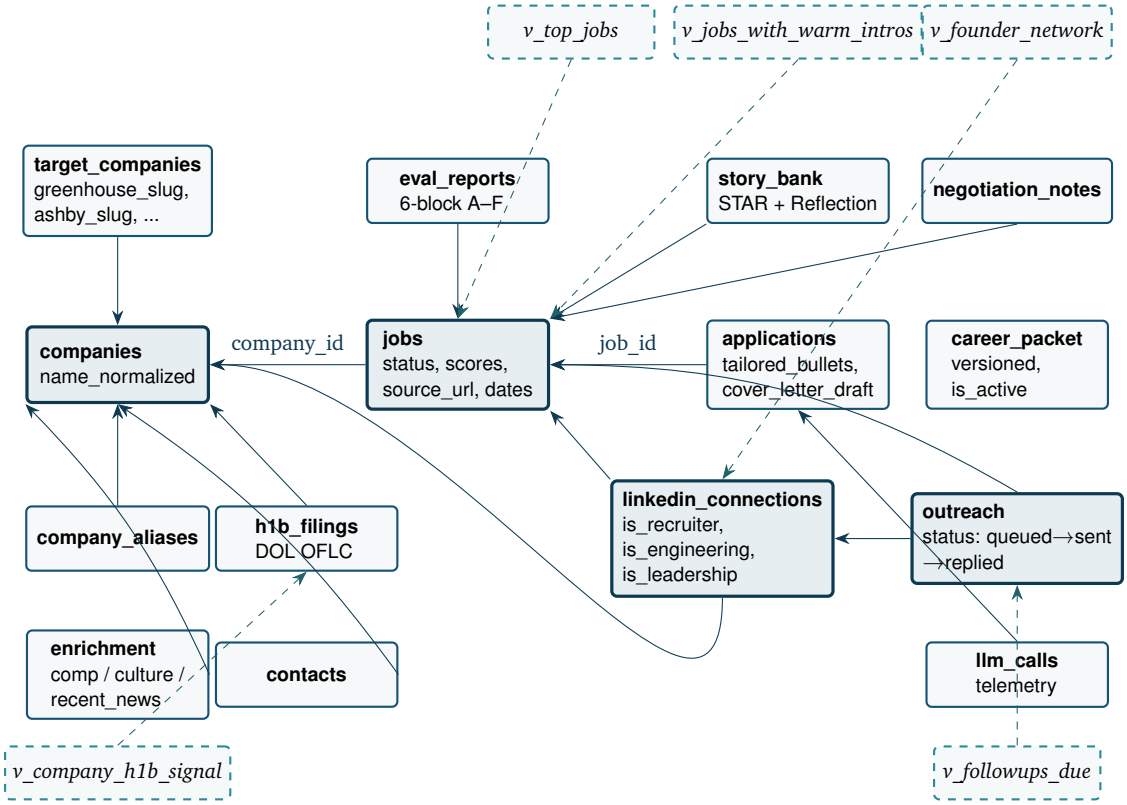
SAFETY: Personal data

Your SQLite DB now contains LinkedIn connections and H1B filings indexed against companies you've rated. The `data/` directory is gitignored by default. Don't commit it. Don't share it.

4 The data model

One SQLite file, 18 tables, 8 views. Two diagrams: the relational shape, and the job lifecycle.

4.1 Relational shape



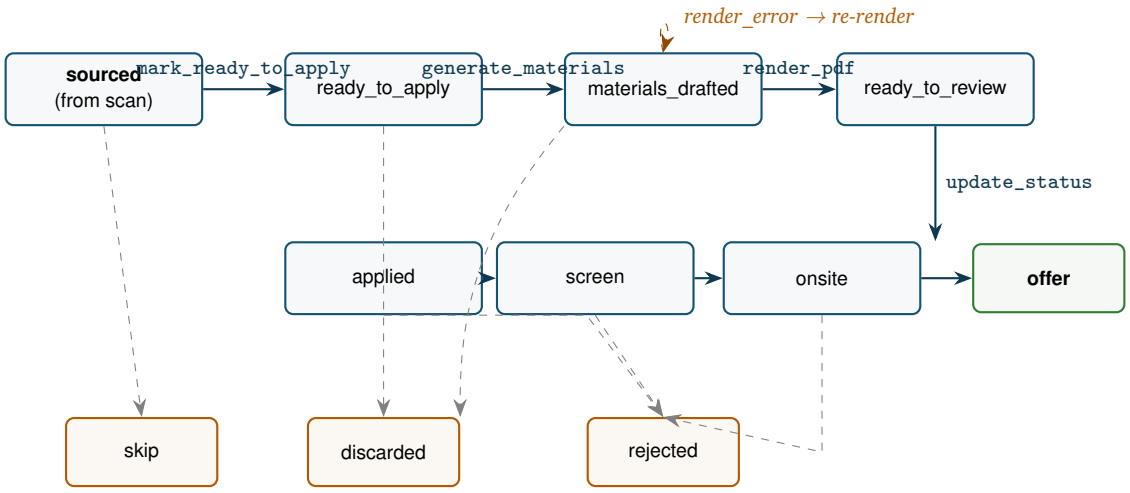
Bold-bordered tables are the ones you'll touch most often via tools. *Italic dashed boxes* are views — they're virtual; the tool layer reads from them. Lines are foreign keys; arrows point at the referenced side.

A few non-obvious shapes:

- **Cross-source dedup.** `jobs.content_hash` (sha-256 of `company + title + location`) is `UNIQUE`. The same role posted on Greenhouse and a company's own careers page collapses to one row.
- **Versioned career_packet.** `update_career_packet` bumps a new row with `is_active = 1` and demotes the previous active row. The history is retained — you can audit which packet generated which materials via `applications.materials_v`.
- **H1B fuzzy join.** `h1b_filings.employer_name_raw` may not match a company's canonical name. `company_aliases` captures legal-name variants; `visa_signal` resolves via the alias table when the direct join misses.
- **Strict-JSON parse log.** Every LLM call lands in `llm_calls` with `parse_ok = 0` and the raw text on failure. `cost_estimate` surfaces parse-error counts so you can tune prompts when a provider drifts.

4.2 The job lifecycle (state machine)

Every job lives in exactly one of these states. The transition rules are enforced by the `update_status` tool and a SQL CHECK constraint; the dashboard at `/` shows the current count per state.



The happy path is left-to-right across the top. **Green** is the only success terminal. **Orange** are the three discard terminals — all reachable from any state via `update_status`. The dashed arrows are weak transitions: you can drop out anywhere, but you can't normally go backwards (re-running `generate_materials` nulls the PDF cache and moves you back to `materials_drafted` for re-render).

5 Tool reference

51 tools, grouped by what they do. For each: what it does, key parameters, what it returns. The default mode for tools that take one is `chat` — the chat client is the brain; `api` mode is the LLM-direct path and needs an API key.

Evaluation

`evaluate_job`
Two-step JD evaluator.

- **Step 1:** pass `input` (a URL or pasted JD text). Returns `job_id`, the normalized JD, the rubric, the report format, and the active career packet for the chat to score.
- **Step 2:** pass `job_id` + `report` (6 blocks) + `scores`. Persists the eval row, updates job scores, returns the report URL.
- `mode:` `api` runs both steps server-side via the configured LLM, with strict-JSON parsing.

`batch_evaluate`
(api only) Rates all unrated jobs through the LLM. Returns A–F tier distribution + parse-error count. Strict-JSON — on parse failure, score_total stays NULL with the raw text logged.

`get_top_jobs`
Triage view. Filter by `min_score` (default 75), `role_category`, `status`, `limit`. Returns per-job:

scores, role, seniority, location, source URL, and the eval report link.

`evaluate_training`
Score a course / cert input on `skill_gap_fill`, `market_signal`, `time_to_value`, with a recommendation.

`evaluate_project`
Score a portfolio-project idea on `resume_signal`, `interview_story_value`, `effort_weeks`, `shipping_risk`.

Materials

`generate_materials`
Picks bullets / tagline / projects from `career_packet.md` per the JD via the tailoring rules. Writes `applications.tailored_bullets + cover_letter_draft`, validates against the visa rail before persisting. Two-step chat shape; `api` mode auto-validates LLM output.

`render_pdf`
Produces resume + cover in any subset of `{pdf, tex, docx}` via the `formats` param (default `["pdf"]`). PDF is HTML→Chromium; `.tex` is a self-contained pdflatex source you can edit and recompile; `.docx` is generated with the docx library for Word / Google Docs editing. All three share one content snapshot taken when the call starts: `cv.md + profile.yml`, the active career packet (chat edits count — no sync-back needed), and the job's persisted tailored materials at their current `materials_v`. URLs persist onto the application row's `rendered_files` JSON. The visa-leakage rail runs against every output before any file is written. Files land under `/files/pdfs/`, `/files/tex/`, `/files/docx/`.

`get_report`
Returns the localhost URL + metadata for the latest `eval_report` of a `job_id`.

Sourcing

`scan_portals`
Fans out across every enabled entry in `portals.yml`. 8 providers: `greenhouse`, `ashby`, `lever`, `workday` (JSON APIs); `amazon` (JSON); `google + playwright_generic` (Playwright). Optional `sources / companies` / title + location filters. Returns counts (found / new / dupes) and the top 25 new rows.

Tracker

`get_tracker`
Filtered/paginated pipeline view as JSON — the same query that powers the dashboard at / (shared `core/tracker_query.ts`). Params: `statuses[]/status`, `min_score/max_score`, `company` (contains), `role_category`, `seniority`, `q` (title/company search), `sort` (`score/discovered/company`) + `dir`, `limit/offset`, `show_trashed`. Returns `items + total_matching` (across all pages) + `counts_by_status` (the FULL pipeline, independent of the filter). Trashed excluded by default. e.g. *"applied jobs scored over 70 with 'engineer' in the title, page 2."*

`update_status`
Move a job to a new status. Stamps `applied_at` on the `applied` transition. Optional `note` appended

to `score_detail.status_history`.

`mark_ready_to_apply`
Bulk-marks job IDs as `ready_to_apply`. Skips terminal-state rows.

`delete_jobs`
Soft-delete 1..N jobs to the trash (by `job_ids` and/or `statuses`, e.g. all `skip`/`discard`). Recoverable; trashed jobs drop out of the tracker, `get_top_jobs`, and batch rating but are retained. Echoes title + company. Never a hard delete.

`restore_jobs`
Move trashed jobs back to their prior state.

`list_trashed`
List currently-trashed jobs (title, company, score, prior status, when trashed) for review before restore/purge.

`purge_jobs`
Hard delete of trashed jobs only — the sole path to permanent removal. `job_ids` or `purge_all: true` (which requires `confirm: true`). Writes a timestamped backup of the affected rows to the project root *before* deleting; echoes what was permanently removed.

TIP: Trash workflow — chat + UI, one shared implementation

Soft-delete is the default destructive action everywhere; hard removal is always explicit, confirmed, and backup-first. In the tracker UI (`/`) the **Status** cell is an inline dropdown and each row has a **Trash** button; the dedicated `/trash` page lists trashed jobs with *Restore*, per-item *Delete permanently* (confirm), and *Empty trash* (confirm + backup). The UI calls `/api/*` endpoints that share the exact `core/job_trash.ts` logic the chat tools use — so a job trashed in chat appears on `/trash` and vice-versa.

TIP: Tracker filtering, search & pagination

At 1000+ jobs the dashboard paginates *server-side* (SQL `WHERE/LIMIT/OFFSET + COUNT` — it never ships every row). Combinable filters (multi-status, min/max score, company-contains, role, seniority, show-trashed), debounced title/company search, sortable Score/Company/Discovered headers, and a page-size selector (25/50/100). All state lives in the URL query string (`/?status=applied&min_score=70&q=engineer&page=2`) so a view is shareable and survives refresh. The summary cards stay **true full-pipeline totals**, independent of the active filter/page. The same `core/tracker_query.ts` backs `get_tracker`, so chat can request the same slices.

Outreach

`find_warm_intros`
Joins `jobs` × non-recruiter LinkedIn connections at the same company. Filter by `company` substring and `min_score`. Sorted by score then contact priority (engineering peers → leadership → others).

`find_founders`
Founders / CEO / CTO / c-suite from your network, derived via `v_founder_network`. Filter by `kind`. Stealth-company rows are flagged but excluded by default.

`add_contacts`
Insert/update **1..N** network contacts from chat in one transaction — complements the bulk `import_linkedin` CSV path, for people found mid-search. Upserts into `linkedin_connections` (matched on `linkedin_url`, else `full_name` + company — no silent duplicates), resolves company names with the same alias normalization, infers `is_recruiter/is_engineering/is_leadership` from the title unless you pass them, and merges (never clobbers) on update. Only `full_name` is required; partial contacts are stored and the per-contact result reports what was missing/unresolved. Invalid rows (no `full_name`) are skipped + reported, not failing the batch. The client parses free-text/pasted info into the structured fields first.

`export_contacts`
Writes ALL contacts + every field (flags, notes, email, resolved company, ids, archived state) to timestamped `contacts_export_*.csv` and `.json` in the project root. The backup / portability path.

`import_contacts`
Imports a `.json`/`.csv` (e.g. a prior export). **Upsert/merge, never delete-and-replace**: blank/absent fields don't overwrite richer existing data; matches existing rows (no duplicates); inserts new. Idempotent re-import. Writes a backup first.

`delete_contacts`
Soft-deletes 1..N contacts (by `id / linkedin_url / full_name`+company). Archived rows are hidden from `find_warm_intros/find_founders` but recoverable in the row. Writes a backup first; echoes exactly which rows matched (name + company + url) so a wrong fuzzy match is catchable.

`draft_outreach`
Two-step chat: step 1 returns connection + job context + outreach tone rules; step 2 takes the chat-drafted `message` and persists it after validation (char cap, no emojis, no exclamation marks, no "refer me", no visa mentions). `api` mode does the LLM call inline.

`draft_followup`
1–2 line nudge for sent-but-unanswered DMs. Same safety rails. Capped at 300 chars.

`draft_reply`
Contextual draft when someone has replied. Takes the received reply text, returns a tone-matched response for human editing. Cap 800 chars.

`get_outreach_queue`
Filter by status (default: `queued / drafted / edited`). Joins connection + company + linked job.

`update_outreach`
Move an outreach row to a new status. Stamps `sent_at` on `sent`; sets `followup_due_at = now + 5d` unless overridden. Re-validates `edited_message` against the safety rails.

`get_followups_due`
Returns outreach in status `sent` with `followup_due_at <= now`.

Visa (optional, can be hidden globally)

`visa_signal`
H1B friendliness band per company: `strong | mixed | weak | none`. Computed from filings count + recency. **Internal only** — never surfaced in any resume, cover letter, or outreach.

`import_linkedin`
Bulk-load `linkedin_connections` from the standard LinkedIn `Connections.csv` export. `dry_run`

flag.

`import_h1b`
Bulk-load `h1b_filings` from a DOL OFLC quarterly LCA CSV. Maps employer names to companies via the alias table; unmatched names persist with `employer_id = NULL`.

> When `JOBOPS_VISA_SCORING=false`, all three visa tools are hidden from `tools/list` and the score formula renormalizes to `0.6 · resume + 0.4 · taste`. See §8.

Interview / offer

`extract_stories`
Parses STAR + Reflection rows out of the latest `eval_report.block_interview` for a job and appends to `story_bank`. Pass an explicit `stories` array to skip parsing.

`get_story_bank`
Returns accumulated stories, with an optional `competency` substring filter on the tag array.

`negotiation_brief`
Returns the negotiation playbook + the most recent `comp` enrichment for the company + a draft framework (anchor / pillars / knobs / hard rules).

Profile + ops

`get_career_packet`
Returns the active packet (id, version, markdown content, source-cv hash).

`update_career_packet`
Replaces the active packet with a new version. Bumps `version`, demotes the previous `is_active`.

`reseed_career_packet`
Rebuilds the packet from the current `cv.md` + `config/profile.yml`. Same as the CLI `reseed` command. Bumps version, demotes the previous active row (history kept). Use after editing `cv.md`. Refuses to seed identity-only unless you pass `confirm_empty_cv: true`.

`edit_packet_item` / `remove_packet_item`
Change or remove ONE item (bullet / project / skill / tagline) in a section (`projects/skills/taglines/education` or a number; experience = 3/4/5), addressed by 1-based index or a matching substring — no whole-document resend. Each versions the packet (history kept); `edit_packet_item` runs the visa-leakage scan on the new text; `remove_packet_item` echoes the removed item.

`restore_packet_version`
With no argument: lists packet versions (number, origin, active, size, notes, time). With `version`: restores that version's content as a new active version — so every granular edit/removal is reversible.

`enrich_company`
Persists a chat- or api-provided enrichment summary for one (company, kind) pair with a 30-day TTL.

`deep_research`
Company brief — aggregates current enrichments + top jobs + warm intros + H1B signal. With `notes` + `kind` provided in `api` mode, summarizes the notes into a single enrichment row.

`daily_digest`
Morning summary — new top jobs since last digest, follow-ups due, recent status changes, cost snapshot. Stamps `digest_state.last_digest_at` unless `dry_run`.

`cost_estimate`
LLM cost aggregated by provider / model / tool over the last N days (default 30). USD estimate based on a built-in rate table.

`doctor`
Read-only health report for the running server — the same checks as the CLI `doctor` command (packet ↔ cv.md sync state, LLM provider/key, sampling + auth posture, active template, modes bundled-vs-overridden, visa scoring, public base URL, Chromium, Node, config files), returned as structured `{ ok, counts, checks[], summary }`. Cold-start-only checks (e.g. the bind target) are relabelled as informational since the server is obviously already bound. Mutates nothing.

Apply (preview only)

`apply_prefill`
Opens the application URL in Chromium, detects common form fields (name, email, phone, LinkedIn, GitHub, cover letter, etc.), drafts values from your profile + tailored materials, takes a full-page screenshot. Returns the preview JSON + the screenshot URL + your resume / cover PDF URLs. **Never submits.**

Scheduler (opt-in cron, off by default)

`scheduler_status`
Lists enabled jobs, available jobs + intervals, and the last fire time.

`scheduler_enable`
Takes an array of job names from: `scan_portals_4h`, `batch_evaluate_30m`, `followups_due_1h`, `daily_digest_morning`. Overrides the previous set.

`scheduler_disable`
Clears the enabled-jobs list. Process keeps running.

6 Worked end-to-end examples

These walkthroughs use a fictional candidate — the new graduate "Casey Riley" targeting Builder PM / Applied AI Engineer roles. Substitute your own profile mentally; the chat client wiring is identical.

6.1 Example A: paste a JD, get a tailored PDF, mark applied

Time required: 8–15 minutes.

> **You:**
>
> I want to apply to this role. Evaluate it, give me a tailored resume + cover, and put it in my tracker.

```
Builder PM / Technical PM - Pinch AI (Remote, US)
We're building developer tools for AI agents and need a hands-on PM who can write PRDs,
ship features end-to-end, and partner with engineering on agentic workflows, evals,
and LLM observability.  4+ years PM, strong SQL + Python, familiarity with LLM systems.
Sponsor visas.  Comp $180-260k.
```

Chat:

Calling `evaluate_job` with the pasted JD...

Server:

Returns `job_id: 9a1b...`, the normalized JD, and three resources to score against (`modes/rubric.md`, `modes/report_format.md`, the active career packet).

Chat:

Reading the rubric and your packet. Scoring:

resume_fit	84	Strong on PRDs, SQL, Python; agentic experience via the `Vector Agents` side project.
taste_fit	90	Lean team, AI tooling, ships fast — matches `narrative.likes`.
visa_fit	88	Explicit sponsorship; comp band well above the OPT floor.
score_total	87	$\lceil 0.5 \cdot 84 + 0.3 \cdot 90 + 0.2 \cdot 88 \rceil$ = A-tier.

Drafting the 6 blocks now... calling `evaluate_job` step 2 with `report + scores`.

Server:

Persists the eval report, renders the HTML.


```
{
  "step": "finalized",
  "job_id": "9a1b...",
  "report_id": "c4d2...",
  "report_url": "http://127.0.0.1:7891/files/reports/c4d2....html",
  "tracker_url": "http://127.0.0.1:7891/"
}
```


Chat:

Report ready: `http://127.0.0.1:7891/files/reports/c4d2....html`. Now generating tailored materials. Calling `generate_materials...`

Server:

Returns the career packet + tailoring rules.

Chat:

Picking tagline option D ("Builder PM with engineering teeth"), foregrounding your e-commerce BI ownership and Vector Agents project, dropping the academic projects. Drafting the cover body (282 words). Calling `generate_materials` again to persist... then `render_pdf` kind=both...

Server:


```
{
  "files": [
    {"kind":"resume","url":"http://127.0.0.1:7891/files/pdfs/resume-builder-pm-9a1b.
  pdf","bytes":271769},
    {"kind":"cover", "url":"http://127.0.0.1:7891/files/pdfs/cover-builder-pm-9a1b.
  pdf", "bytes":61464}
  ]
}
```


Chat:

Both PDFs at the links above. Click resume and cover to inspect. When you've submitted via Pinch's portal, say so and I'll update the tracker.

You:

Done, submitted. Mark it applied.

Chat:

Calling `update_status job_id=9a1b... status=applied...` done. Pipeline now shows: **1 applied**, **1 onsite**, **2 ready_to_apply**. Open the tracker at `http://127.0.0.1:7891/`.

EXAMPLE: what just happened

Two MCP tool calls (`evaluate_job` twice, `render_pdf` once, `update_status` once), one transcript, four artifacts: the 6-block report, the resume PDF, the cover PDF, an updated tracker row. The chat did the writing; the server did the persistence + rendering. You did the deciding and submitting.

6.2 Example B: scan portals, triage, batch evaluate

Time required: 5 minutes of human attention, 3 minutes of background LLM work.

You:

Scan my portals and rate everything new. Then show me the top PM jobs.

Chat:

Calling `scan_portals...`

Server:


```
{
  "companies_scanned": 18,
  "jobs_found": 412,
  "jobs_new": 47,
  "jobs_dupes": 5,
```

```
  "sources": ["greenhouse","ashby","lever"],
  "errors": []
}
```


Chat:

47 new jobs after title/location filtering. Now `batch_evaluate` on the unrated set...

Server:


```
{
  "rated": 45,
  "parse_errors": 2,
  "distribution": {"A":4, "B":11, "C":17, "D":10, "F":3, "unrated":2}
}
```


Chat:

A/B-tier yield: 15 out of 47 (32%). Two parse errors will retry on the next batch run. Pulling top PM jobs... calling `get_top_jobs role_category=pm min_score=80`...

Server:


```
{
  "count": 6,
  "items": [
    {"title":"Builder PM","company_name":"Vellum","score_total":91,"location":"Remote
  , US","report_url":"http://.../reports/...html"},
    {"title":"AI Product Manager","company_name":"Mosaic","score_total":88,"location"
  :"NYC","report_url":"http://.../..."},
    {"title":"Technical PM, Agents","company_name":"Loop AI","score_total":86,"
  location":"Remote","report_url":"http://.../..."}
  ]
}
```


Chat:

6 PM jobs ≥ 80. Click any report link to read the 6-block evaluation. Want me to mark the top 3 `ready_to_apply` and start generating materials?

EXAMPLE: the triage instinct

The brief 5-minute version is enough. You don't need to read all 45 reports. Skim the top 3–5, kill the false positives with `update_status status=skip`, and queue the keepers for tomorrow's material-generation pass. The whole point of the rubric is to push the human-attention budget toward the top of the funnel.

6.3 Example C: find a warm intro, draft outreach, follow up

Time required: 4–7 minutes per DM. Prereq: `import_linkedin` run once.

You:

I scored 91 on the Vellum Builder PM role. Do I know anyone there?

Chat:

Calling `find_warm_intros company=Vellum...`

Server:


```
{
  "count": 2,
  "items": [
    {"full_name":"Priya Subramaniam","position":"Staff Engineer, Agents","
  contact_priority":1,"connection_id":"abc-1","linkedin_url":"https://linkedin.com/in
  /priya-test"},
    {"full_name":"Jordan Lee","position":"Head of Product","contact_priority":2,"
  connection_id":"abc-2","linkedin_url":"https://linkedin.com/in/jordan-test"}
  ]
}
```


Chat:

Two: Priya (Staff Eng) and Jordan (Head of Product). Priya's the engineering-peer angle — the tone rules say lead with a technical question. Calling `draft_outreach connection_id=abc-1 job_id=<vellum> type=warm_intro_request...`

Server:

Returns connection + job context + the active career packet + `modes/outreach_tone.md`.

Chat:

Drafting at 287 chars (cap is 600). Reviewing against the rails: no emojis, no exclamation marks, no "refer me", no visa mention, soft close. Calling `draft_outreach` again with the `message` to persist...

Server:


```
{
  "outreach_id": "o-12",
  "status": "drafted",
  "validation": {"ok": true, "message_len": 287, "limit": 600, "type": "warm", "
  issues": []}
}
```

Chat:

Drafted. Here's the message — edit before sending:

> saw your recent post about agent eval harnesses — curious how y'all are thinking about multi-step trace replay. i've been shipping agentic workflows on AutoGen + LlamaIndex at my last role, and the trace problem is the one we kept hitting. would love a 15-min chat if it lines up. no worries if not a fit.

When you send, tell me and I'll update the row + queue a follow-up for 5 days out.

You:

Sent.

Chat:

`update_outreach id=o-12 status=sent` — done. Follow-up due 5 days from today.
(5 days later, no reply.)
`get_followups_due` shows o-12. Calling `draft_followup id=o-12`...

Server:

Returns the original message + days-since-sent for context.

Chat:

Drafting 1-line nudge (98 chars):

> still curious about your take on trace replay — happy to come back if it ever makes sense.

EXAMPLE: tone rules at work

The draft passed all rails: under 600 chars, no exclamation marks, no emoji, no "refer me", no visa or sponsorship reference, leads with a specific technical observation, drops one credibility marker ("shipping agentic workflows on AutoGen + LlamaIndex"), and closes soft. The follow-up nudge keeps the same tone in one line. Both will sound like a peer-to-peer DM, not a job application.

7 The daily operating ritual

TIP

This is the most important section of the guide. The system pays back in repeated use, not one-shot setup. A polished evaluation that never becomes a sent application is worth nothing.

A realistic split for someone running a serious search — **10–30 minutes a day**, three time-boxed slots.

Morning: triage (10 minutes)

1. Open the chat. "*Run `daily_digest`.*"
2. Read the digest: new top jobs since yesterday, follow-ups due today, status changes in the pipeline.
3. For each new A-tier job: open the report link, skim the 6 blocks. **One of three actions:** `mark_ready_to_apply`, `update_status status=skip`, or do nothing (decide tomorrow). *Do not let A-tier jobs sit in `sourced` for more than 48 hours — they go stale.*

Midday: send (15 minutes)

1. "*`get_followups_due`.*" For each row: `draft_followup`, edit, send, `update_outreach status=sent`.
2. "*`find_warm_intros` for the top 3 `ready_to_apply` jobs.*" For each non-trivial match: `draft_outreach`, edit, send, `update_outreach status=sent`.
3. Anyone replied? "*Show outreach in `replied` status.*" For each: `draft_reply`, edit, send.

Evening: apply (15 minutes)

1. Pick **one** job from `ready_to_apply`. `generate_materials`. Review the tailored bullets in chat.
2. `render_pdf kind=both`. Open both URLs, eyeball the PDF.
3. "*`apply_prefill` for this job.*" Open the screenshot URL, see the draft form values. Open the actual application URL in a new tab, paste + upload, submit manually.
4. `update_status status=applied`. Done for the day.

Weekly: review (20 minutes, e.g. Sunday)

1. "*Show the tracker.*" Eyeball the funnel — where are you stuck?
2. "*Show the story bank.*" Did this week's interviews add new stories? Tag them by competency.
3. "*`cost_estimate`.*" Pennies for the LLM. Sanity-check before the next batch.
4. Edit `modes/rubric.md` or `config/profile.yml` if last week's matches were systematically off. The system gets sharper with iteration.

> **The bottleneck is human attention, not the tool.** 30 well-targeted applications with warm intros over 4 weeks outperforms 200 cold blasts. Resist the temptation to optimize the system instead of using it. If you find yourself fiddling with the rubric weights instead of clicking *send*, that's the failure mode — close the IDE, open the chat.

8 Customization

Every behaviour-shaping piece is a Markdown file or a config entry. No code changes required.

Editing the modes

`init` scaffolds the six mode files into `<project-root>/modes/` (alongside `cv.md` / `config/profile.yml` / `portals.yml`) so they're yours to edit — you never touch the package install. The loader reads your project-root copy *first* and falls back to the bundled default for any file you haven't overridden. A re-`init` never clobbers an edited copy: it warns and keeps your edits. `doctor` reports which mode files are user-overridden versus bundled defaults.

These files are exposed as MCP resources and live-reloaded on edit (the server watches both the project-root and bundled directories). After saving, the chat picks up the change on the next `resources/read` — no restart.

File	What it controls
`modes/rubric.md`	Scoring dimensions, weights, role-priority order, archetype-override rule, hard rules
`modes/report_format.md`	6-block A–F (+G) report shape
`modes/tailoring_rules.md`	Resume/cover tailoring decisions per `role_category`
`modes/outreach_tone.md`	Warm-intro / founder DM char caps, forbidden phrases, persona rules
`modes/negotiation_playbook.md`	Negotiation framework + scripts (anchor, pillars, knobs)
`modes/career_packet.md`	Your bullet/project bank — the superset of every claim

Per-archetype taglines

Section 2 of the career packet is a set of one-line positioning statements — one per role archetype you target — from which the rater picks the best fit per JD. Rather than re-stamp those by hand after every reseed, declare them once in `config/profile.yml` under a `taglines:` block:

```
taglines:
  "Builder PM":          "ships product with engineering teeth"
  "Applied AI Engineer": "turns LLM prototypes into shipped systems"
  "Forward-Deployed":    "embeds with customers and closes the deal"
```

On the next `reseed` (CLI `npx @mohith_das/jobops reseed` or the `reseed_career_packet` tool) these auto-populate Section 2 — one bullet per archetype. The field is backwards-compatible: omit it entirely and Section 2 keeps the editable template placeholders exactly as before.

Company alias / fuzzy matching

Company names arrive in different legal forms across data sources — a LinkedIn connection at "Anthropic", an H1B filing under "ANTHROPIC PBC", a JD scraped as "Anthropic, Inc.". The server normalizes all of them by stripping common legal suffixes (`Inc, LLC, PBC, Ltd, Corp, Co, GmbH, ...`), lowercasing, and trimming punctuation, then matches on that canonical key. The result: all three resolve to the *same* `companies` row, so the `company_id` joins that `find_warm_intros` and `visa_signal` depend on actually line up. This applies consistently to `import_linkedin`, `import_h1b`, JD ingestion, and `visa_signal`. Resolved variants are recorded in the `company_aliases` table for provenance.

Disabling visa scoring (fully optional)

Visa signal exists because the system's original author was on OPT/STEM-OPT and needed to filter for visa-friendly employers. If you don't need that — US citizens, non-US users, anyone scoring roles where sponsorship is irrelevant — set:

```
export JOBOPS_VISA_SCORING=false
```

When disabled, the server:

- Drops `visa_fit` from the rubric. `score_total = round(0.6 · resume_fit + 0.4 · taste_fit)` — enforced server-side regardless of what the chat returns.
- Prepends a "VISA SCORING DISABLED" override block to the top of the `jobops://modes/rubric` resource.
- Hides `visa_signal`, `import_h1b`, `import_linkedin` from `tools/list`.
- Strips `score_visa_fit` from `get_top_jobs` items and the eval-report HTML badge.
- Records `visa_scoring_enabled: false` in every `score_detail` blob so you can audit which formula scored a given row.

Every other feature is unaffected.

Adding target companies

Append to `portals.yml tracked_companies:` — restart the server (or wait for the next `scan_portals` call which re-reads the file).

```
- name: "Vellum"
  careers_url: "https://job-boards.greenhouse.io/vellum"
  priority: 1
  enabled: true

- name: "ExampleCo"
  workday_url: "https://examplco.wd5.myworkdayjobs.com/External"
  priority: 2
  enabled: true

- name: "ClosedBoardCo"
  provider: "playwright_generic"
  careers_url: "https://example.com/careers"
  selectors:
    item:     "a.job-card"
    title:    ".job-title"
    location: ".job-location"
    wait_for: ".jobs-loaded"
  enabled: false
```

Non-US markets

The defaults assume nothing. To localize:

- Translate the headings of `modes/rubric.md` and `modes/negotiation_playbook.md`; keep the formula structure.
- Update `narrative.likes / dislikes` and `compensation.target_range` in `config/profile.yml` to match your market.
- Add region-specific portals to `portals.yml`. The generic Playwright provider works against any HTML careers page given the right selectors.
- Localize `title_filter.positive / negative` — "Ingeniero" for Spanish-language postings, "Ingénieur" for French, etc.

Tuning the rubric weights

Edit `modes/rubric.md`. Change the weights paragraph and the formula. Next `evaluate_job` call returns the new rubric; the chat will use the updated math.

TIP

If you change weights mid-search, your existing `score_total` values reflect the old formula. `batch_evaluate` will re-rate any job whose `score_total` is NULL — set the column to NULL for everything you want re-scored.

Custom resume / cover themes

`render_pdf` accepts a `template` argument so you can swap the bundled default for a theme you author. A theme is a directory under `templates/themes/<name>/` that holds any of `resume.tex`, `cover.tex`, `resume.html`, `cover.html`; each file is a plain template with `{{PLACEHOLDER}}` slots the renderer fills with the same tailored content it uses for the default theme. See `TEMPLATES.md` in the repo root for the full placeholder reference.

To author a custom theme:

```
mkdir -p ~/job-themes/compact
# Author resume.tex / cover.tex / resume.html / cover.html -- copy the
# bundled default as a starting point and edit the preamble or layout.

export JOBOPS_TEMPLATE_DIR=~/job-themes
npx @mohith_das/jobops templates              # lists bundled + user themes

# Then in your MCP chat, pass template= to render_pdf:
#   render_pdf job_id=... kind=both formats=["pdf","tex"] template="compact"
```

The loader checks `$JOBOPS_TEMPLATE_DIR` first, so a `default/` directory in your themes dir overrides the bundled default. To make a custom theme the implicit default for every `render_pdf` call (instead of passing `template=...`):

```
export JOBOPS_DEFAULT_TEMPLATE=compact
```

Hard rules still apply regardless of theme: the visa-leakage scan runs on `cover_body` *before* substitution, so a custom template cannot inject visa/work-auth language. A theme that omits a placeholder silently drops that section (graceful degradation); in `.tex` themes, commenting a placeholder out (`% {{SUMMARY_SECTION}}`) drops the section the same way — substitution skips

LaTeX comments. A theme missing `\documentclass` or `\begin{document}` produces an error that quotes the theme name + file path — pdflatex's own backtrace never reaches the user.

`.docx` is generated programmatically and does not use themes. If you need visual variation for `.docx`, edit the output in Word.

9 MCP protocol features: sampling, elicitation, auth

Three capabilities lean on the MCP protocol itself. All are **capability-gated**: the server checks what the connected client advertises during `initialize` and only uses a feature when it is supported, so older clients degrade gracefully to the pre-existing file/key paths.

SAFETY: Sampling depends on your client — most don't support it

Sampling and elicitation are server→client *requests* that the client must opt into by advertising the capability in its `initialize` handshake. **The transport is not sufficient on its own:** stdio is *necessary* (the stateless streamable-HTTP transport can't deliver a server-initiated request at all), but a stdio client still has to advertise `sampling`. **Most clients do not — including Claude Desktop, as of now** (it advertises only its UI extension, never `sampling`). So on Claude Desktop, server-side scoring uses the **BYO key**, which is expected and correct. Check current support at modelcontextprotocol.io/clients, and run the `doctor` tool to see the live negotiated state for your client.

MCP sampling — no LLM key required *when the client supports it*

The `api`-path scorers (`batch_evaluate`, `evaluate_job` `mode="api"`) will use **sampling**: the server issues a `sampling/createMessage` request to the connected client and the *client's own model* produces the completion — same rubric, same strict-JSON contract as the BYO-key path. One `Completer` abstraction (`src/core/scoring.ts`) backs both, and `evaluate_job`/`batch_evaluate` report `scored_via: "sampling"` or `"api"`. This engages **automatically if (and only if) a sampling-capable client connects** — no configuration needed.

- **Selection order:** sampling (only if the client advertised `sampling` *and* `JOBOPS_SAMPLING` ≠ `false`) → BYO key (`JOBOPS_LLM_PROVIDER` + key) → otherwise `evaluate_job` `mode="chat"` still works (the chat scores it). On a client without sampling (e.g. Claude Desktop), set the BYO key for `batch_evaluate` / `mode="api"`.
- **Cost:** when sampling *is* used it runs on the client's model, so the client bears the cost. `cost_estimate` records every sampling call flagged `cost_borne_by: client` with a $0 server estimate, so the total is not misread.

MCP elicitation — structured input without YAML edits

`update_profile` uses **form-mode elicitation**: the server sends a JSON-schema form (identity fields + up to three archetype/tagline pairs); on `accept` it merges the values into `config/profile.yml` and reseeds the career packet in one step. No client elicitation support? Pass the same values via the `fields` argument, or edit the YAML by hand — both still work.

For **sensitive inputs** (a LinkedIn export path, credentials) the server uses **URL-mode elicitation**

(2025-11-25): it hands the client a one-time local URL where you enter the value directly into your own server, so the value *never passes through the MCP client or the chat transcript*. `import_linkedin` uses this automatically when you omit `path` and the client supports it; otherwise pass `path` as before.

Auth — the remote / PII surface

See §10 below for the full table. In short: localhost-only stays open and frictionless; binding to anything else requires `JOBOPS_AUTH_TOKEN` or the server refuses to start.

10 Security: auth + the PII surface

The HTTP server exposes PII — resume PDFs, cover letters, eval reports, your LinkedIn connections, H1B-derived employer data. The auth posture is a pure function of the bind host and whether a token is set (`src/core/auth.ts`, `resolveAuthPolicy`):

Bind host	Token	Result
`127.0.0.1` (default)	unset	**Open** — frictionless local use; PII stays on loopback.
`127.0.0.1`	set	**Token required** — bearer auth enforced even locally (opt-in).
non-localhost	**unset**	**Refuses to start** (default-deny).
non-localhost	set	**Token required** — bearer auth on every PII route.

When a token is required, `/mcp`, `/files/*`, and the dashboard (`/`) demand `Authorization: Bearer <token>`. A missing/invalid token returns `401` with a `WWW-Authenticate` header pointing at `/.well-known/oauth-protected-resource`. This realizes the MCP 2025-06-18 "server as OAuth Resource Server" model to the extent practical for a self-hosted single-user tool: one operator-provisioned static token (`JOBOPS_AUTH_TOKEN`), no full authorization-server flow.

```
export JOBOPS_HOST=0.0.0.0
export JOBOPS_AUTH_TOKEN="$(openssl rand -hex 32)"
export JOBOPS_PUBLIC_BASE_URL="https://jobs.example.ts.net"
npx @mohith_das/jobops start
```

Protected: `/mcp`, `/files/*`, `/`. **Open by design:** `/healthz` (liveness, no PII) and the discovery metadata. The URL-mode elicitation capture form (`/elicit/:id`) is gated by an unguessable one-time id rather than the bearer token, so you can complete it in a browser. **Hard rule:** PII must never be served unauthenticated to a network — the default-deny boot guard makes a missing token fail loudly instead of silently exposing your data. Even with a token, prefer a private network (Tailscale / VPN / authenticated reverse proxy) over the public internet.

11 The hard rules (safety model)

Five rules are enforced at the code level, not just documented. Removing any of them is a one-line change you'd notice in code review.

SAFETY: Rule 1: Visa data never reaches a candidate-facing artifact

`scanForVisaLeakage()` runs inside `render_pdf` (on `cover_body`) and inside `generate_materials` (on the entire serialized output) before persistence. If a leak is detected the call **fails**, not warns. Visa data is internal to `visa_signal` scoring and `visa_fit` — that's the only valid surface.

SAFETY: Rule 2: Never invent claims outside the career packet

The materials generator instruction set explicitly tells the chat: "NEVER invent metrics not in the packet." The packet is the authoritative source; bullets that cite numbers not present in it should fail human review. `update_career_packet` bumps a version so the audit trail is permanent.

SAFETY: Rule 3: Human-in-the-loop everywhere

No tool auto-submits an application. `apply_prefill` opens the page, drafts values, takes a screenshot, and stops — the human submits manually. No tool auto-sends a DM. `draft_outreach` persists a draft; the human sends, then calls `update_outreach status=sent`. Removing this gate is the single change that would turn this from a tool into a spam cannon.

SAFETY: Rule 4: Strict-JSON parsing with visible failure

Every `api`-path LLM call uses `response_format: json_object`. On parse failure the call records `parse_ok = 0` + the raw text in `llm_calls`, and the score row stays NULL — *never* a default `(0,0,0,0)`. "Silent zeros" was the JSA pipeline's worst failure mode; `cost_estimate` surfaces parse-error counts so drift is visible.

SAFETY: Rule 5: Serialized writes

Every tool that mutates the tracker / applications / outreach goes through `runInWriteLock()` in `src/db.ts`. Concurrent tool calls won't interleave their writes; the state machine stays consistent.

SAFETY: Rule 6: PII is never served unauthenticated to a network

Binding to a non-localhost host without `JOBOPS_AUTH_TOKEN` makes the server **refuse to boot** (default-deny). When a token is set, `/mcp`, `/files/*`, and the dashboard require a bearer token; only `/healthz` + discovery metadata stay open. Resume PDFs, LinkedIn connections, and H1B data never reach an unauthenticated remote caller. See §10.

The outreach safety rails extend Rule 1 with four more pattern groups, validated in `validateOutreach()` before any DM is persisted:

Rule	What it rejects
`char_cap`	Warm: 600 chars. Founder: 300. Follow-up: 300. Reply: 800.
`no_visa_mentions`	H1B / OPT / EAD / visa / sponsorship / green card / work permit / citizenship.
`no_refer_me`	"refer me" / "put my resume in front of" / "good word" / "get me an interview".
`no_emojis`	Any emoji codepoint.
`no_exclamation_marks`	Any `!`.
`no_cliches`	"I hope this finds you well" / "I'd love to pick your brain".

A failing draft is returned to the chat with the specific rule + offending hit so it can be revised. The drafter never silently strips and saves.

12 Troubleshooting and FAQ

Setup issues

***"Chromium isn't installed"* on first `start`**

`npx @mohith_das/jobops start` auto-installs Chromium on first run (150 MB, 30 s). If you skipped it via `-skip-chromium-check` or the install errored, run `npx playwright install chromium` manually and re-try.

***"Tool not found"* from the chat client**

Your client connected before the server had registered tools. Check the connection mode — for Claude Desktop, the `command: npx` block should spawn the server inline with the `-stdio` flag; for LibreChat, the server must already be running on the URL you specified. Run `npx @mohith_das/jobops doctor` to confirm.

LibreChat shows "connected" but every call fails

You're in Docker, the LibreChat container can't reach your host's `127.0.0.1`. Swap to `host.docker.internal` **and** add the entry to `mcpSettings.allowedAddresses`. On Linux, add `extra_hosts: ["host.docker.internal:host-gateway"]` to docker-compose.yml.

LLM tools error with "No LLM provider configured"

Set `JOBOPS_LLM_PROVIDER` + the matching API key. The default `chat` mode of every tool works without one — only `batch_evaluate` and `mode: api` need a key.

Migrations didn't run

The server applies pending migrations on first DB open. If the `data/` directory is on a read-only filesystem, the boot will error. Set `JOBOPS_DATA_DIR` to a writable path and re-run.

***Doctor says "cv.md missing"* but it's in the directory**

You probably launched from a different CWD than where `init` ran. Set `JOBOPS_PROJECT_ROOT` to the absolute path, or `cd` into the right directory before `start`.

Conceptual FAQ

Does this work without visa data?

Yes. `JOBOPS_VISA_SCORING=false` drops the visa surface entirely — formula renormalizes, tools hidden, columns stripped. Use the system exactly as documented; only the visa-specific bits go quiet.

Does this work outside the US?
Yes. The defaults assume no specific market. Edit `modes/*.md` to match your local language / comp norms, add region-specific portals to `portals.yml`, and adjust `title_filter`.

How much does an LLM key cost?
On Gemini 2.5 Flash (free tier), a single `batch_evaluate` on 50 fresh jobs is comfortably under the per-minute quota and the per-day quota. `cost_estimate -days 30` reports your actual usage. The chat-mode tools cost zero on the server side (your chat client pays its own inference cost via your subscription).

Why isn't there an `auto_apply` tool?
That's the line. Every irreversible boundary — submit, send DM, post comment — is a human action. The system makes the prep cheaper; the human still decides and acts. See §13.

Can I run two of these on the same machine?
Yes — pick different ports and project roots. Each gets its own SQLite file; the two never interact.

Where does the SQLite file go?
By default, `<project_root>/data/jobops.db`. Override with `JOBOPS_DATA_DIR`.

How big does the DB get?
After 6 months of daily use with 10–20k jobs scored, expect 50–100 MB. SQLite handles that comfortably with WAL mode. Back up by copying the directory; nothing else.

Will the schema change?
Yes, additively. Future migrations land in `src/migrations/00X_....sql` and apply on first DB open after `npm update`. The migration ledger lives in the `_migrations` table so already-applied ones aren't re-run.

13 This finds jobs. It doesn't send them.

A frank coda.

`jobops` is good at four things:

1. **Sourcing.** Polling 20+ ATS endpoints in a single `scan_portals` call, with cross-source dedup. Better than refreshing tabs by hand.
2. **Scoring.** A consistent rubric applied to every JD, persisted with the reasoning. Better than re-deciding "is this worth my time?" a hundred times.
3. **Drafting.** Tailored bullets, ATS-clean PDFs, on-tone DMs, follow-up nudges. Better than starting from scratch each time.
4. **Tracking.** One database, one tracker URL. Better than a spreadsheet that drifts.

It is **not** good at — and is intentionally architected against — the thing that actually closes the loop: *deciding which roles to apply to and sending the application or the DM*. That is the human's job, every day. The system can score 50 jobs in an evening; you can probably honestly apply to two or three a day if the materials are tailored and the outreach is real. The bottleneck is the human, by design.

If you find yourself **tweaking the rubric instead of clicking send**, the system is failing you — not because it's broken, but because it's optimizing the wrong axis. The daily ritual in §7 is built around that risk. Use it.

Good luck out there.